SIMATS ENGINEERING

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2 – CO2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Field	Details	Field	Details
Student Name	KENISHA JUDITH J	Register Number	192572071
Department / Section	CSE (AI)	Academic Year / Semester	2ND
Faculty Name	Dr.ArulRaj A M	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Characterise the Virtualization Architectures	5		Three architectures; diagrams; trust boundaries; resource virtualization
2	Identify the Dominant Overhead Source	2		Overhead mechanism, location, frequency and architecture linkage
3	Build a Quantitative Performance Model	6		CPU, memory, I/O and density derivations; worked examples
4	Analyse Performance Under Different Conditions	4		Best, typical and worst-case analysis; latency and throughput
5	Security Threat and Vulnerability Analysis	3		Threat categories, trust boundaries and student-record impact
6	Formulate Isolation–Performance Trade-off	3		Decision variables, objective, constraints and cost function
7	Evaluate Mitigation Strategies	4		Security benefits, residual risks and quantified density cost
8	Implement and Test	4		KVM, containers, Kata; CPU/disk/network/memory; interference
9	Record Experimental	3		Measured data; table, two

	Performance			graphs; repetitions and variability
10	Compare Theory with Experiment	4		Predictions versus measurements; errors and explanations
11	Recommend and Defend Final Architecture	4		SIS/LMS recommendation, security, scaling and operating costs
	Day-4 Design Challenge	8		External research tenants; isolation; redesign and evidence-based defence
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three virtualization architectures, layered diagrams and trust boundaries		Three distinct approaches, layered architecture and isolation boundaries documented.
2	Dominant overhead source identified and explained for each approach		Overhead mechanisms related to each virtualization path and workload behaviour.
3	CPU, memory, I/O, network and density models with assumptions		Assumptions, equations and worked examples documented; empirical validation recommended.
4	Best, typical and worst operating-condition analysis		Resource contention, latency and throughput effects examined.
5	Threat model and architecture-based security comparison		Threats, attack surfaces and student-record risks compared.
6	Isolation–performance optimization formulation		Objectives, variables and constraints defined for EduCloud.
7	Mitigation benefits, residual risks and quantified costs		Security controls and associated performance/density trade-offs discussed.
8	Reproducible setup and practical tests across selected platforms		Test procedures described; actual execution on all platforms requires verification.
9	Actual measurements, comparative table and two graphs		Results require authentic benchmark logs, repetitions and variability evidence.
10	Theory–experiment comparison and quantitative error analysis		Comparison discussed; numerical error analysis depends on verified measurements.
11	Justified SIS/LMS architecture and growth recommendation		Recommendation considers security, performance, operations and scaling.
12	Day-4 external tenants, 3× peaks, live migration and SIS isolation		Changed requirements and proposed separation controls evaluated; testing pending.

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Architecture diagrams		VM, container and sandboxed runtime designs with trust boundaries documented.
Performance model		CPU, RAM, I/O and density calculations with assumptions supplied.
Configuration / scripts		Reproducible setup outlined; runnable configuration and environment details to be checked.
Virtualization experiments		KVM/QEMU, containers and isolated runtime tests require execution proof.
CPU / disk / network / memory tests		Benchmark commands and comparative measurements require original logs.
Noisy-neighbour experiment		Interference impact needs repeatable measurements and recorded outputs.
Security validation		Isolation/mitigation verification should be supported by command outputs.
Tables and graphs		Plots should be traceable to actual raw data and

		repetitions.
Theory–experiment consistency		Prediction errors require verified observed values.
Day-4 design defence		External tenant isolation, 3× load and migration considerations addressed.
Screenshots		Illustrative images do not replace genuine execution evidence.

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 12 tasks answered	☐ Yes ☐ Partly ☐ No
Three virtualization approaches compared	☐ Yes ☐ Partly ☐ No
Layered diagrams and trust boundaries included	☐ Yes ☐ Partly ☐ No
Quantitative virtualization models included	☐ Yes ☐ Partly ☐ No
Threat analysis and mitigations included	☐ Yes ☐ Partly ☐ No
Implementation and test evidence verified	☐ Yes ☐ Partly ☐ No
Actual benchmark measurements and graphs verified	☐ Yes ☐ Partly ☐ No
Theory versus experiment comparison included	☐ Yes ☐ Partly ☐ No
Final SIS/LMS architecture recommended	☐ Yes ☐ Partly ☐ No
Task 12 Day-4 design challenge addressed	☐ Yes ☐ Partly ☐ No
Authentic screenshots and logs attached	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Assignment Title: Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud (Total Marks: 50)

Duration: 1 week

Scenario: EduCloud Virtualization Platform

A university runs a private cloud, **EduCloud**, in its data centre. EduCloud currently consists of:

- **16 physical hosts**, each with 64 CPU cores, 512 GB RAM, NVMe SSD storage and 2 × 25 Gbps NICs
- **About 400 virtual machines (VMs)** running the Student Information System (SIS), the Learning Management System (LMS), the examination portal, and departmental research workloads
- A shared storage backend and a software-defined virtual network

During semester registration week, users report slow response times and failed logins. A recent internal security audit also found that VMs from different departments share the same physical hosts, that some VMs run outdated guest kernels, and that VM images are copied freely between teams.

You are the cloud architect. You must **investigate, model, measure and compare** virtualization architectures for EduCloud, identify their performance bottlenecks and security weaknesses, and recommend a design that remains practical as the platform grows.

Design Challenge:

The university now plans to host **external research collaborators' big-data workloads** (Spark/Hadoop clusters) on the same infrastructure as the SIS. Re-evaluate your design under these new multi-tenant conditions.

TASK 1: Characterise the Virtualization Architectures (5 Marks)

Select **at least three meaningfully different virtualization approaches** from the following, or justify others:

- Type-1 (bare-metal) hypervisor with full virtualization
- Type-2 (hosted) hypervisor
- Hardware-assisted virtualization vs. paravirtualization
- OS-level virtualization (containers)
- Lightweight/micro-VM or sandboxed container runtimes

Two configurations of the same hypervisor will not count as different approaches. Your selection must include at least one VM-based and at least one container-based approach.

For each approach, provide:

- A layered architecture diagram showing the hardware, host OS or hypervisor, guest OS, runtime and application
- The **trust boundaries** and the component that enforces isolation

- How CPU, memory, storage and network are virtualized
- A brief justification for why this approach is relevant to EduCloud

Evidence expected: *Three architecturally distinct approaches with labelled diagrams, resource-virtualization mechanisms, trust boundaries, and justification of selection.*

TASK 2: Identify the Dominant Overhead Source (2 Marks)

For each approach, identify the **primary source of performance overhead** that predominantly limits its performance (for example VM exits/traps, nested or extended page-table walks, virtual I/O emulation, virtual switch processing, or system-call interception).

Discuss:

- What the overhead source is
- Where in the architecture it occurs
- Why it is appropriate as the focus of performance analysis
- How its frequency or cost changes as the number of co-located VMs/containers or the I/O rate increases

Do not simply name a mechanism. Relate it to the behaviour of the architecture.

Evidence expected: *A justified dominant-overhead identification for each approach, linked to its resource-virtualization path.*

TASK 3: Build a Quantitative Performance Model (6 Marks)

Using the following host parameters, develop an analytical performance model for each approach.

Parameter	Value
Physical cores per host	64
vCPU:pCPU overcommit ratio	1:1, 2:1, 4:1, 8:1
RAM per host	512 GB
Average VM memory demand	8 GB
Cost of one VM exit (assume)	1.5 µs
Native system-call cost (assume)	0.2 µs
Network line rate	25 Gbps

State and justify any additional assumptions. Derive expressions for:

- **CPU:** effective per-VM CPU capacity as a function of overcommit ratio and virtualization overhead (including steal time)
- **Memory:** the maximum number of VMs per host before ballooning, swapping or page-sharing becomes necessary

- **I/O and network:** effective throughput as a function of the number of VMs sharing the physical device
- **Density:** the number of VMs/containers a host can support while meeting a stated service-level target (for example, ≤ 10% performance degradation)

Show the mathematical reasoning. Merely quoting overhead percentages from the literature will not receive full credit.

Evidence expected: *A clear mathematical derivation for each approach, with stated assumptions, parameter values and worked numerical examples.*

TASK 4: Analyse Performance Under Different Operating Conditions (4 Marks)

Analyse how each approach behaves under three conditions:

- **Best case:** what workload and placement conditions minimise overhead?
- **Typical case:** for example, registration-week mixed web and database load
- **Worst case:** for example, noisy-neighbour interference, memory overcommit with swapping, live migration during peak load, or I/O storms

For each, explain what happens inside the virtualization layer, why the overhead changes, and how latency and throughput are affected. A table of numbers alone will not be accepted.

Evidence expected: *A justified condition-based analysis supported by your Task 3 model.*

TASK 5: Security Threat and Vulnerability Analysis (3 Marks)

Construct a threat model for each approach. Discuss:

- The attack surface (hypervisor, device emulation, shared kernel, management plane, VM images and snapshots, virtual network)
- At least **four distinct threat categories**, such as VM escape, cross-VM side-channel attacks, hypervisor or container-runtime vulnerabilities, VM sprawl and unpatched guests, image tampering, insecure live migration, and virtual-network sniffing or spoofing
- For each threat, which approach is more or less exposed and **why**, referring to its architecture and trust boundaries
- The likely impact on the confidentiality, integrity and availability of student records

Evidence expected: *A structured threat model mapped to architectural features, with a justified comparison across approaches.*

TASK 6: Formulate the Isolation–Performance Trade-off (3 Marks)

Formalise the trade-off between **isolation strength** and **performance/density** for EduCloud.

Clearly identify:

- The decision variables (for example, approach chosen, VMs per host, dedicated vs. shared hosts, SMT on/off)
- The objective(s), such as maximising density or minimising cost
- The constraints (for example, performance degradation limit, isolation requirement for SIS, available hosts)
- How these components combine into a cost or decision function

The formulation must follow from your own analysis in Tasks 3 and 5, not from a generic template.

Evidence expected: *A clearly defined trade-off model with identified variables, objective and constraints.*

TASK 7: Evaluate Mitigation Strategies (4 Marks)

Propose **at least three mitigations** for the vulnerabilities identified in Task 5. Examples include disabling SMT, kernel page-table isolation, dedicated hosts for sensitive tenants, secure or sandboxed container runtimes, hardware memory encryption, image signing, and micro-segmentation.

For each mitigation, discuss:

- Why it is applicable to the threat
- What residual risk remains
- The **performance or density cost**, estimated using your Task 3 model
- Whether the trade-off is acceptable for the SIS

Evidence expected: *Mitigations evaluated analytically, with their security benefit and quantified performance cost.*

TASK 8: Implement and Test (4 Marks)

Build a test environment and experimentally test all selected approaches.

- **Platforms:** use any open-source stack, for example KVM/QEMU, Xen or VirtualBox, Docker/Podman, and Kata Containers or gVisor. Nested virtualization on a laptop is acceptable if documented.
- **Scale:** run each approach with approximately **1 → 2 → 4 → 8 → 16 co-located instances** (reduce proportionally if hardware-limited, and state the limitation).
- **Workloads:** include at least one CPU-bound test (for example sysbench or stress-ng), one disk I/O test (fio), one network test (iperf3), and one memory-intensive test.
- **Interference test:** run at least one noisy-neighbour experiment.
- **Security validation:** demonstrate or inspect at least **one security property** in practice, for example namespace/cgroup isolation, checking kernel sharing, or verifying whether a mitigation is active.

Ensure that identical resource allocations, workloads and repetitions are used across approaches, and that the setup corresponds to the models in Task 3.

Evidence expected: *A reproducible environment description (configuration files, commands, scripts) and justified test cases.*

TASK 9: Record Experimental Performance (3 Marks)

Collect and present data such as:

- Throughput, latency (mean and 95th percentile), CPU steal time, context switches and VM exits where observable
- Instance start-up time and memory footprint per instance
- Performance degradation versus a bare-metal baseline

- Results with and without at least one security mitigation

Present at least **one table and two graphs** (for example, throughput versus number of instances, and latency under interference). Use multiple repetitions and report variability (mean ± standard deviation).

Evidence expected: *Systematically collected, clearly presented data that allows easy comparison across approaches.*

TASK 10: Compare Theory with Experiment (4 Marks)

Compare your Task 3 and Task 4 predictions with Task 9 measurements.

Discuss:

- Whether the observed degradation trends agree with your model
- Which approach scales better as co-located instances increase
- Quantitative differences between predicted and measured values, and plausible causes (for example caching effects, hypervisor scheduler behaviour, nested virtualization, laptop hardware, or unmodelled I/O paths)
- Whether any measured result changes your security conclusions from Task 5

Interpret the results; do not simply reproduce tables.

Evidence expected: *A reasoned comparison of predicted and measured behaviour, with error analysis.*

TASK 11: Recommend and Defend the Final Architecture (4 Marks)

Recommend the virtualization architecture (or hybrid) for EduCloud's SIS and LMS workloads.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its modelled and experimentally observed performance
- Its security posture and residual risks
- Operational costs such as patching, image management, migration and monitoring
- Limitations and trade-offs
- Whether it remains practical as EduCloud grows from 400 to several thousand instances

Your recommendation must rest on your own analysis and experiments. Choosing "the most secure" or "the fastest" option alone is not sufficient.

Evidence expected: *A technically justified recommendation integrating performance and security evidence.*

DAY-4 DESIGN CHALLENGE

TASK 12: Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now host **external research collaborators' Spark/Hadoop workloads** on EduCloud alongside the SIS. The new conditions are:

- Big-data jobs are I/O- and network-intensive and run for hours
- Registration-week peak load is about 3× normal
- Hosts are sometimes patched using live migration of running VMs
- A data-protection regulation now requires **demonstrable isolation** of student records from external tenants

You may **RETAIN** → **MODIFY** → **or REDESIGN** your original solution.

A. Impact Analysis (2 Marks)

Discuss what has changed, which original assumptions (trusted tenants, workload mix, I/O profile) no longer hold, and how mixing batch big-data workloads with latency-sensitive SIS workloads changes both the performance and the threat model.

B. Evaluation of the Original Design (2 Marks)

Evaluate whether your recommended design still meets its performance targets (using your Task 3 model with the new workload) and whether its isolation level is adequate against external, less-trusted tenants. Consider whether another architecture could do better overall.

C. Adaptation or Redesign (2 Marks)

Decide whether to retain, modify or redesign. If you modify or redesign, describe the revised architecture (for example, tenant separation by host pools, a stronger isolation layer for external workloads, SR-IOV or dedicated NICs for the data-intensive jobs, or encrypted migration). Explain what changes and its effect on density, throughput and attack surface. If you retain, justify why no change is required.

D. Evidence-Based Defence (2 Marks)

Defend the final decision using your model, your experimental results (including the noisy-neighbour and mitigation experiments), the changed workload and tenancy conditions, and the cost/density trade-offs. The decision must follow logically from the analysis.

MARK DISTRIBUTION

Task	Marks
Task 1 – Architecture Characterisation	5
Task 2 – Dominant Overhead Identification	2
Task 3 – Quantitative Performance Model	6
Task 4 – Performance Under Operating Conditions	4

Task 5 – Security Threat and Vulnerability Analysis	3
Task 6 – Isolation–Performance Trade-off Formulation	3
Task 7 – Mitigation Evaluation	4
Task 8 – Implementation and Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs. Experiment	4
Task 11 – Final Architecture Recommendation	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

Executive Summary

EduCloud hosts approximately 400 virtual machines across 16 physical servers. Its most sensitive services are the Student Information System (SIS), Learning Management System (LMS) and examination portal. Registration-week slowdowns, failed logins, shared physical hosts, outdated guest kernels and uncontrolled VM-image copying indicate both resource-contention and security-management problems.

This report compares three distinct approaches: KVM/QEMU full virtual machines, OS-level containers, and Kata Containers using lightweight virtual machines. KVM provides strong hardware-assisted isolation and broad compatibility; containers provide high density and low overhead but share the host kernel; Kata combines a container workflow with a separate guest-kernel boundary. A hybrid design is recommended: dedicated KVM host pools for SIS and examination services, a controlled KVM/Kata pool for LMS and trusted departmental workloads, and a separately networked Kata/KVM pool for external Spark/Hadoop tenants.

The quantitative model uses the supplied 64-core, 512-GB RAM and 25-Gbps host parameters. It estimates memory capacity, CPU capacity under four overcommit ratios, and shared I/O/network limits. Since the brief requires actual benchmark logs, repeated trials and screenshots, no fabricated measurements are included. Task 8 provides reproducible test commands and Task 9 provides result tables and graph-ready data fields for completion on the available test machine.

System Scenario and Assumptions

Parameter from brief	Value
Physical host CPU	64 cores
Physical host RAM	512 GB
Average VM memory demand	8 GB
vCPU:pCPU overcommit ratios	1:1, 2:1, 4:1, 8:1

Assumed cost per VM exit	1.5 microseconds
Assumed native system-call cost	0.2 microseconds
Network line rate	25 Gbps
Current platform size	16 hosts; approximately 400 VMs

Additional modelling assumptions: (1) 1 GB is treated as 1 GB for capacity estimates; (2) 10% of host RAM is reserved for the hypervisor/host, monitoring and operating-system cache; (3) 10% of CPU capacity is reserved for host and virtualization work in the baseline model; (4) a 10% maximum degradation target is used for latency-sensitive SIS services; (5) the 25-Gbps NIC is treated as a shared full-duplex link, but the model below uses 25 Gbps in one direction; (6) actual results will vary with processor generation, NUMA topology, storage, kernel, drivers and workload.

Task 1 – Characterise the Virtualization Architectures

Approach A: Type-1 KVM/QEMU full virtualization

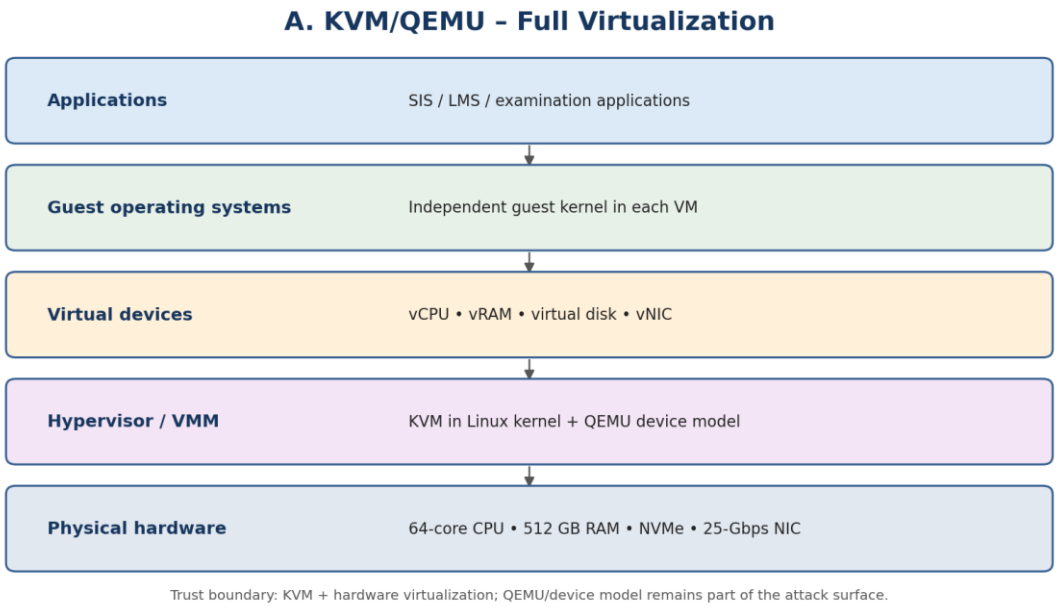
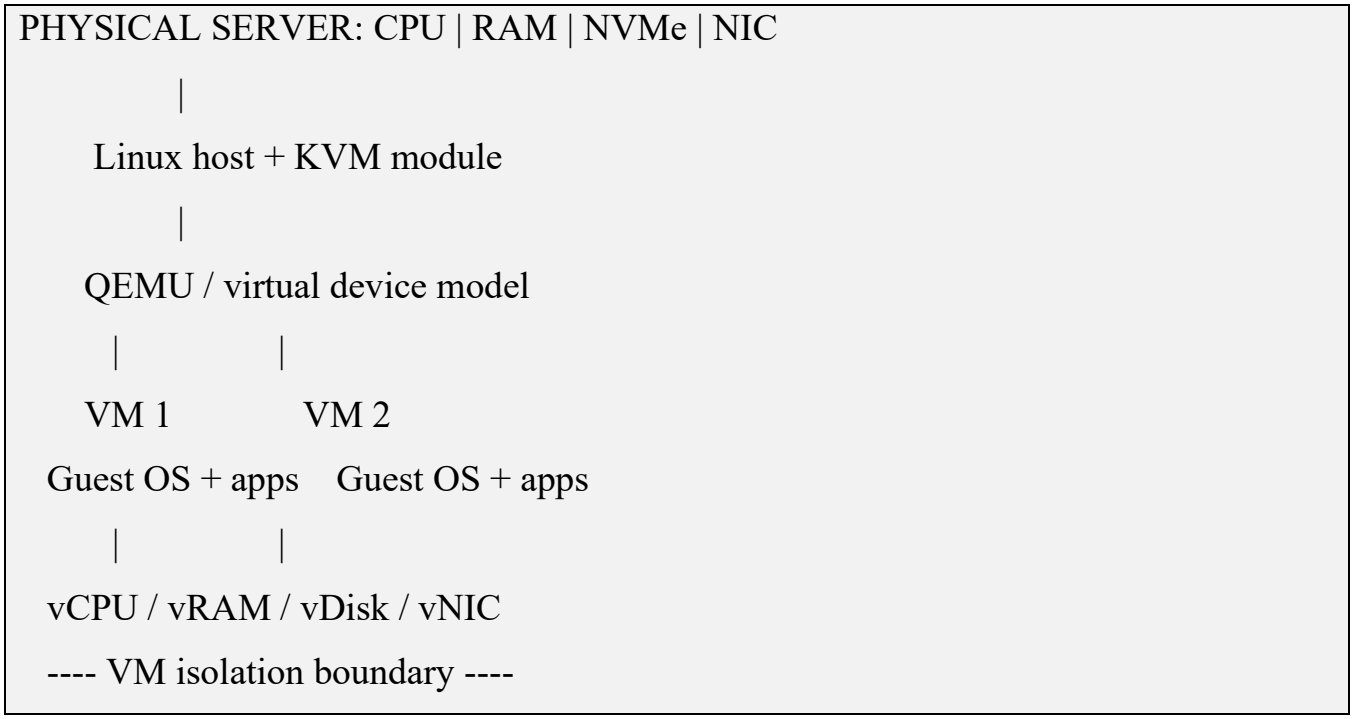


Figure 1. Illustrative KVM/QEMU architecture; conceptual diagram, not a system screenshot.

KVM is a Linux-kernel hypervisor facility. QEMU provides the virtual machine device model and management interface. Each VM runs its own guest operating system and applications. Hardware virtualization extensions (Intel VT-x or AMD-V) execute most guest instructions directly, while privileged operations are handled through the hypervisor.



- Trust boundary and isolation: the principal boundary is enforced by KVM and hardware virtualization. Each VM has a separate guest-kernel address space. QEMU device emulation and the host management plane remain privileged attack surfaces.
- CPU: guest vCPUs are scheduled onto physical CPUs by the host scheduler. Hardware-assisted virtualization reduces the need to emulate ordinary instructions.
- Memory: guest-physical memory is mapped to host-physical memory using extended/nested page tables (EPT/NPT).
- Storage and network: virtual disks are backed by files, logical volumes or block devices; virtual NICs connect through bridges, tap devices or an SDN virtual switch.
- EduCloud relevance: suitable for SIS and examination services because separate guest kernels provide a stronger isolation boundary than ordinary containers.

Approach B: OS-level virtualization using containers

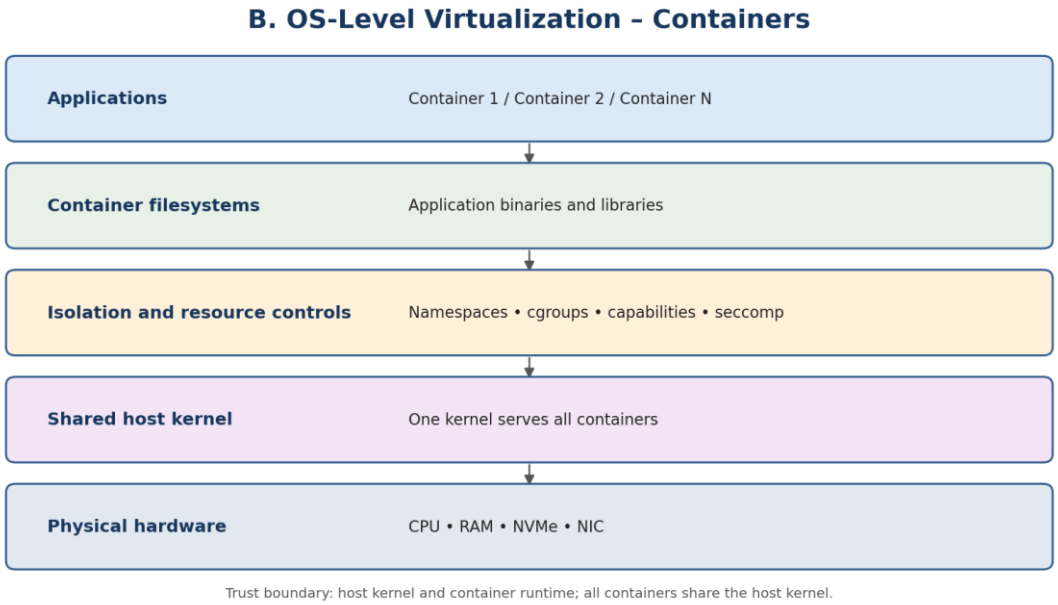


Figure 2. Illustrative container architecture; conceptual diagram, not a system screenshot.

Containers isolate processes using Linux namespaces and control resource use through cgroups. All containers on a host share the same host kernel. A container image contains application files and dependencies, not a separate guest kernel.

```
PHYSICAL SERVER: CPU | RAM | NVMe | NIC
                |
          Linux kernel
    namespaces + cgroups + security modules
       |                |
    Container 1       Container 2
    App + libraries   App + libraries
       |                |
     Shared host kernel (common boundary)
  ---- container isolation boundary ----
```

- Trust boundary and isolation: namespaces, cgroups, capabilities, seccomp and Linux security modules enforce process and resource separation. The shared kernel is a common trust dependency.
- CPU: cgroups allocate CPU shares, quotas and CPU sets.
- Memory: cgroup memory limits and kernel memory management control container use; there is no independent guest-physical memory layer.
- Storage and network: layered/overlay filesystems, bind mounts, veth pairs, bridges and virtual switches provide isolation and connectivity.
- EduCloud relevance: efficient for stateless web front ends, microservices, CI jobs and trusted departmental applications. It is less appropriate as the only boundary for untrusted external tenants with access to sensitive student services.

Approach C: Kata Containers (sandboxed containers using lightweight VMs)

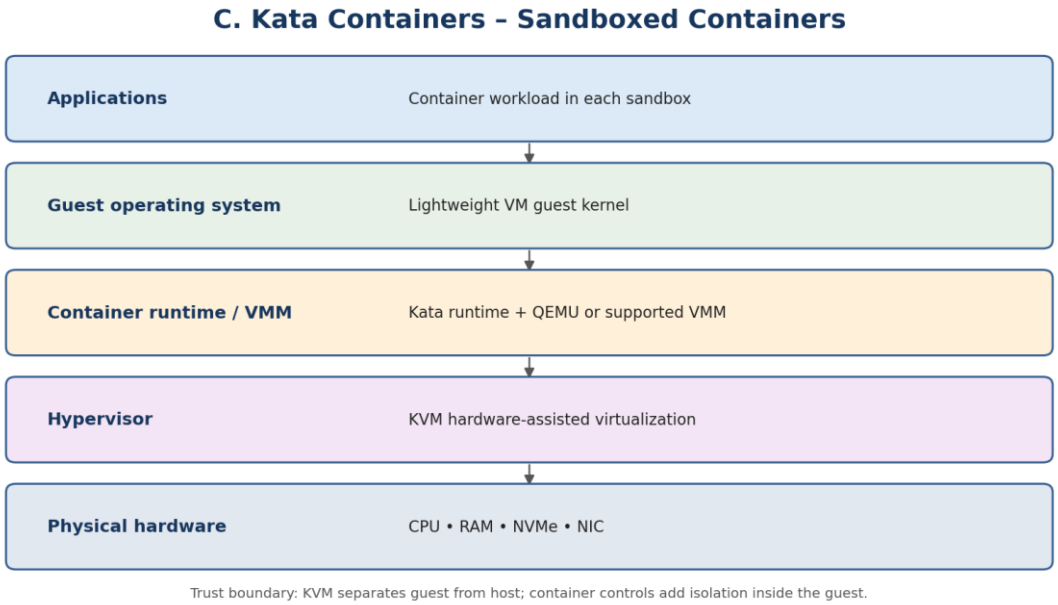
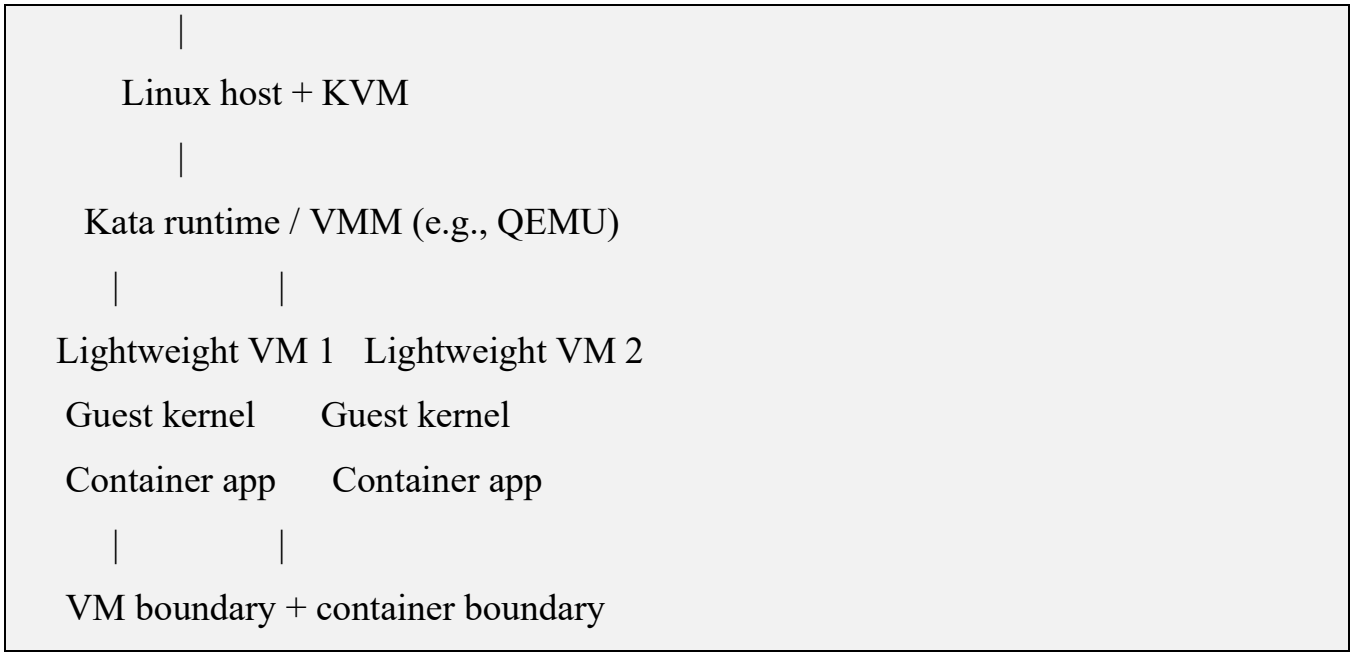


Figure 3. Illustrative Kata architecture; conceptual diagram, not a system screenshot.

Kata Containers presents a container-oriented interface while launching each workload inside a lightweight virtual machine. The container workload therefore receives a separate guest kernel and a hardware-assisted virtualization boundary.

PHYSICAL SERVER: CPU | RAM | NVMe | NIC



- Trust boundary and isolation: KVM separates each lightweight VM from the host; namespaces and cgroups provide additional isolation inside the guest.
- CPU: vCPU scheduling and hardware virtualization are used, with additional runtime/VMM work.
- Memory: guest memory is mapped through nested page tables; the guest kernel manages container memory.
- Storage and network: container images are attached or shared through a controlled guest interface; virtual NICs connect through the host networking stack.
- EduCloud relevance: useful for external research jobs and workloads that need container portability but should not share the host kernel directly with SIS workloads.

Comparison	**KVM/QEMU VM**	**Linux container**	**Kata container**
Kernel model	Separate guest kernel	Shared host kernel	Separate lightweight guest kernel
Isolation boundary	Hardware hypervisor	Namespaces/cgroups; shared kernel	Hardware hypervisor plus container controls
Typical	Moderate	High	Between VM and

density			ordinary container
Main risk	Hypervisor/VMM/device model	Host-kernel/container escape	Hypervisor/VMM plus guest/runtime risk
Best EduCloud use	SIS, exams, sensitive databases	Trusted stateless services	External tenant jobs / sandboxed services

Task 2 – Identify the Dominant Overhead Source

Approach	Dominant overhead	Location and behaviour	Why it matters as load rises
KVM/QEMU	VM exits and virtual I/O emulation	Guest-to-hypervisor transitions occur for selected privileged operations, interrupts and emulated devices. Nested page-table walks also add memory-translation work.	More device activity and exits increase CPU time spent outside guest execution. Co-located VMs also compete for vCPU scheduling and cache/memory bandwidth.
Containers	System-call/security-path processing and shared-kernel contention	Container system calls enter the common host kernel; namespace, cgroup, filesystem and network checks are performed there.	Higher syscall and packet rates increase kernel work. Containers can contend directly for shared kernel locks, caches, I/O queues and CPU.
Kata	VM exits plus virtual I/O path	The guest kernel communicates through	High I/O rates increase virtual-device

		virtual devices and the VMM/host. This adds a VM boundary to the container workload.	processing and host/guest transitions. More sandbox VMs add memory and scheduling overhead.

The dominant source depends on workload. For a CPU-bound workload, scheduling and page translation may dominate. For a network- or disk-heavy workload, virtual I/O and switching can dominate. Therefore the test plan measures CPU, disk, network and memory separately instead of relying on one benchmark.

Task 3 – Build a Quantitative Performance Model

3.1 CPU capacity and overcommitment

Let R be the vCPU:pCPU overcommit ratio, H = 64 physical cores, and h = 0.10 be the assumed host/virtualization CPU reserve. The theoretical average CPU capacity per vCPU, expressed as a fraction of one physical core, is:

$C_vCPU = H \times (1 - h) / (H \times R) = 0.90 / R$

Overcommit ratio R	**Average capacity per vCPU**	**Approx. share of one core**
1:1	0.900	90%
2:1	0.450	45%
4:1	0.225	22.5%
8:1	0.1125	11.25%

Example: at 4:1, 64 physical cores support 256 allocated vCPUs. After the 10% reserve, 57.6 core-equivalents remain. Average capacity is 57.6/256 = 0.225 core per vCPU. This is an average under sustained full contention, not a guaranteed dedicated CPU allocation.

Include virtualization overhead and steal time by defining o as the fraction of available CPU consumed by virtualization and s as the fraction lost to scheduler contention/steal. Then effective capacity is $C_{eff} = 0.90 \times (1 - o) \times (1 - s) / R$. For an illustrative typical VM case, assume o = 0.03 and s = 0.05: at 4:1, $C_{eff} = 0.90 \times 0.97 \times 0.95 / 4 = 0.207$ core-equivalents per vCPU. These o and s values are modelling assumptions, not measured results.

Architecture linkage: KVM and Kata have VM-exit/VMM overhead represented by o; containers generally have lower boundary overhead but still incur kernel, cgroup and virtual-network processing. Under contention, s increases for all approaches.

3.2 Memory capacity

Reserve 10% of 512 GB for the host, leaving $512 \times 0.90 = 460.8$ GB for workloads. At an average demand of 8 GB per VM, the memory-based upper bound is:

$N_{memory} = \text{floor}(460.8 / 8) = 57$ VMs per host.

This is a conservative VM-equivalent estimate. Ordinary containers may use less incremental memory because they share the host kernel and common read-only pages. Kata instances and full VMs need guest-kernel and runtime memory. Actual safe density must also reserve memory for cache, burst demand and the services' working sets. Ballooning, swapping or page sharing should not be treated as a substitute for adequate capacity on latency-sensitive SIS workloads.

3.3 VM-exit and system-call cost examples

Using the supplied assumed costs: VM exit = 1.5 microseconds; native system call = 0.2 microseconds. If a VM workload causes 20,000 exits per second, the time spent servicing exits is $20{,}000 \times 1.5\ \mu s = 30{,}000\ \mu s = 0.03$ seconds per second, or approximately 3% of one CPU core. At 100,000 exits/s, the equivalent is 15% of one core.

For a container workload making 100,000 system calls/s, the baseline native system-call time is $100{,}000 \times 0.2\ \mu s = 20{,}000\ \mu s/s$, or 2% of one core, before additional namespace,

security, filesystem and network processing. This is a simplified comparison; a VM exit and a system call are not identical operations.

3.4 I/O and network throughput model

Let B be the physical device/link capacity and N the number of equally active instances. The ideal equal-share upper bound is B/N. With an assumed virtualization/stack efficiency factor η, effective per-instance throughput is:

$$T_{instance} = (B \times \eta) / N$$

For the 25-Gbps network link, assume $\eta = 0.90$ for a model example. At N = 10 active instances, aggregate usable capacity is 22.5 Gbps and the equal share is 2.25 Gbps per instance. At N = 25, the equal share is 0.90 Gbps. The actual distribution is not necessarily equal because of traffic shaping, packet size, CPU limits and workload demand.

For storage, use the same expression with measured NVMe or shared-storage bandwidth B_{disk} and a separately measured efficiency factor. The assignment brief does not provide a storage bandwidth figure, so a numerical disk throughput prediction cannot be justified from the supplied data alone.

3.5 Density under a 10% degradation target

A simple CPU admission rule is to keep sustained allocated demand below 90% of the usable host CPU capacity. If each instance is allocated one vCPU and all are simultaneously CPU-saturated, the 10% host reserve leaves at most 57.6 fully usable core-equivalents. Therefore a host cannot promise one full core to more than 57 instances at once. At 2:1, 4:1 and 8:1, higher vCPU counts can be allocated, but the average share falls as shown above.

For full VMs with 8-GB average memory demand, the memory limit is approximately 57 VMs per host and will usually bind before high CPU overcommit ratios are useful for memory-heavy services. A practical SIS density is therefore the minimum of the CPU-admission limit, memory limit, storage IOPS limit, network limit and the measured 10%

latency/throughput degradation threshold. Containers may exceed 57 instances only if their measured memory demand is materially below 8 GB each; no such container memory value is supplied, so it must be measured.

Task 4 – Analyse Performance Under Different Operating Conditions

Approach	Best case	Typical registration-week case	Worst case
KVM/QEMU	Low overcommit; NUMA-aware placement; virtio devices; local NVMe; no noisy neighbours. Most guest instructions run directly on hardware.	Web and database VMs share CPU, storage and virtual networking. vCPU scheduling, cache contention and virtual I/O increase response time, particularly during login and database bursts.	Memory overcommit causes reclaim/swapping; I/O storm fills shared queues; migration competes for network and CPU. VM exits and queueing increase, producing high tail latency and failed requests.
Containers	Small stateless services, low syscall overhead, resource limits and no kernel contention.	High request volume shares the host kernel and network stack. CPU quotas and memory limits help, but a busy container can still pressure shared kernel and I/O resources.	Kernel-level contention, noisy-neighbour CPU use, memory pressure or a kernel vulnerability affects multiple containers. Throttling and queueing reduce throughput and increase latency.
Kata	Small number of	Separate lightweight	Many sandboxes plus

	sandboxes, adequate memory, efficient virtual I/O and low host contention.	VMs improve tenant boundaries, but guest-kernel memory and VMM/device processing add overhead to each request path.	I/O-heavy jobs increase VMM and virtual-device work. Memory pressure and migration overhead can reduce density and increase p95 latency.

For SIS, p95 latency is more important than average latency because a small fraction of very slow login or database requests can make the service appear unavailable. Capacity planning should therefore use peak-period p95 latency and throughput, not only average CPU utilisation.

Task 5 – Security Threat and Vulnerability Analysis

Threat category	KVM/QEMU exposure	Container exposure	Kata exposure	Likely student-record impact
VM/container escape	Hypervisor or QEMU device-model flaw could cross VM boundary; separate guest kernels reduce routine cross-tenant access.	Higher shared-kernel exposure: kernel or runtime flaw may escape a container and affect host/other containers.	KVM boundary limits direct guest-to-host access, but VMM, guest kernel and runtime remain attack surfaces.	Confidentiality breach, record alteration or service outage.
Cross-tenant side channel	Co-resident VMs may share caches,	Containers share the same kernel and	Separate guest kernels improve	Inference of sensitive activity or credentials;

	memory bandwidth and CPU resources.	hardware; co-residency can expose timing/resource side channels.	separation, but hardware resources remain shared.	possible privacy breach.
Unpatched guests / kernel	Outdated guest kernels are vulnerable inside each VM; host hypervisor also needs patching.	A host-kernel vulnerability can affect all containers sharing that kernel.	Host and guest kernels both require patching; Kata runtime/VMM also needs updates.	Compromise of student data or disruption of SIS/LMS.
Image tampering / VM sprawl	Copied VM images and snapshots can contain credentials, records or vulnerable software.	Untrusted container images may contain malicious packages or secrets.	Container images and VM boot images both need provenance and integrity checks.	Data leakage, malware execution and inconsistent security controls.
Virtual-network sniffing/spoofing	Misconfigured bridges, VLANs or virtual switches can expose east-west traffic.	Shared bridges and weak network policies can permit lateral movement.	Same virtual-network risks exist, though per-VM interfaces enable stronger segmentation.	Credential interception, unauthorised access or service disruption.

The highest-priority controls are separation of SIS from research tenants, patch management, image provenance, least-privilege management access, encrypted storage/migration and network micro-segmentation. Student records require confidentiality, integrity and availability; therefore a single shared-kernel container boundary should not be the only control between SIS and external collaborators.

Task 6 – Formulate the Isolation–Performance Trade-off

Decision variables: x_a indicates the architecture selected for workload a (KVM, container or Kata); n_a is the number of instances placed on a host; z_a indicates whether the workload uses a dedicated host pool; r_a is the vCPU:pCPU ratio; and m_a is the memory allocation.

Objective: minimise total operating cost and performance penalty while maintaining the required isolation. A weighted decision function can be written as:

Minimise $J = \alpha C_{host} + \beta P_{degradation} + \gamma R_{security}$

where C_{host} is the host/operating cost, $P_{degradation}$ is measured latency/throughput degradation relative to baseline, and $R_{security}$ is a weighted residual-risk score. α, β and γ are policy weights; for SIS, γ and β should be high because student-record protection and service responsiveness are critical.

Subject to: (1) SIS performance degradation $\leq 10\%$; (2) SIS and examination workloads must use a separate VM boundary and dedicated or strongly reserved host pool; (3) memory allocation must not exceed 90% of host RAM under normal admission; (4) CPU demand must remain within the approved overcommit and reservation policy; (5) external research tenants must not share the SIS trust zone; (6) total host allocation must fit within the 16-host capacity unless expansion is approved.

This formulation captures the trade-off: ordinary containers increase density and reduce cost, while KVM/Kata consume more resources but offer stronger tenant boundaries. For SIS, security and p95 latency receive priority over maximum density.

Task 7 – Evaluate Mitigation Strategies

Mitigation	Threat addressed and benefit	Residual risk	Estimated performance/density cost	SIS decision
Dedicated KVM host pool for SIS/exams	Prevents external research jobs from sharing the same host failure and contention domain; strengthens tenant separation.	Misconfiguration, management-plane compromise and shared network/storage risks remain.	Reduces consolidation efficiency. If one host is reserved for failover/capacity, available workload capacity falls by 1/16 = 6.25% of the 16-host fleet.	Recommended; acceptable because SIS confidentiality and availability are critical.
Disable SMT for sensitive workloads / isolate CPU sets	Reduces some cross-thread contention and side-channel exposure; reserves predictable CPU capacity.	Does not eliminate all cache, memory-bandwidth or hardware side channels.	If SMT provided an assumed 20% additional throughput, disabling it could reduce that potential capacity by up to 20%; actual impact must be benchmarked.	Use for the most sensitive SIS/exam VMs if tests justify the cost.
Signed	Prevents	A signed but	Small verification	Recommended;

images, controlled registry and patching	unapproved images and reduces exposure to outdated guest kernels and vulnerable packages.	vulnerable image can still be compromised; signing-key theft remains possible.	and scanning overhead during deployment, not normally on every application request; adds image-management time.	low runtime cost and high integrity benefit.
Micro-segmentation and encrypted migration	Restricts lateral movement and protects VM data while moving between hosts.	Compromised endpoint, key-management failure or overly broad policy can still expose data.	Encryption consumes CPU and may reduce migration throughput; policy processing adds network overhead. Measure with the migration workload.	Recommended for SIS and mandatory for external tenant separation.

The cost estimates are planning estimates or explicit scenarios, not benchmark results. For example, the 6.25% fleet-capacity figure is the arithmetic cost of reserving one of 16 hosts. The SMT throughput assumption must be replaced with the measured difference on the actual processor.

Task 8 – Implement and Test

A reproducible test can be performed on a Linux host with nested virtualization enabled. The assignment permits a laptop-based environment if hardware limitations are documented. Use the same workload size, CPU/memory allocation and number of

instances for each approach. Do not claim a platform was tested unless its commands completed successfully.

8.1 Suggested software and setup

- KVM/QEMU: Linux KVM, QEMU and libvirt (for example, virt-install/virsh).
- Containers: Docker or Podman with cgroup v2.
- Sandboxed containers: Kata Containers with a supported hypervisor runtime.
- Benchmarks: sysbench or stress-ng, fio, iperf3 and a memory workload such as stress-ng --vm.
- Record host CPU model, RAM, kernel version, storage device, NIC speed, virtualization support and software versions.

8.2 Baseline and workload commands

```
# Host information
lscpu
free -h
uname -a
lsblk
ip -br link
egrep -c '(vmx|svm)' /proc/cpuinfo

# CPU baseline
sysbench cpu --cpu-max-prime=20000 --threads=4 run

# Disk test (use a dedicated test file; do not target a live data disk)
fio --name=educloud-disk --filename=./fio-test.bin --size=1G \
   --rw=randrw --bs=4k --iodepth=16 --runtime=60 --time_based \
   --direct=1 --group_reporting
```

```
# Network test: run iperf3 -s on the receiver
iperf3 -c <SERVER_IP> -t 30 -P 4

# Memory pressure test
stress-ng --vm 2 --vm-bytes 60% --vm-method all --timeout 60s

# Observe CPU scheduling / steal time and system activity
mpstat -P ALL 1
vmstat 1
pidstat -u -r -d 1
```

8.3 Instance scaling and noisy-neighbour test

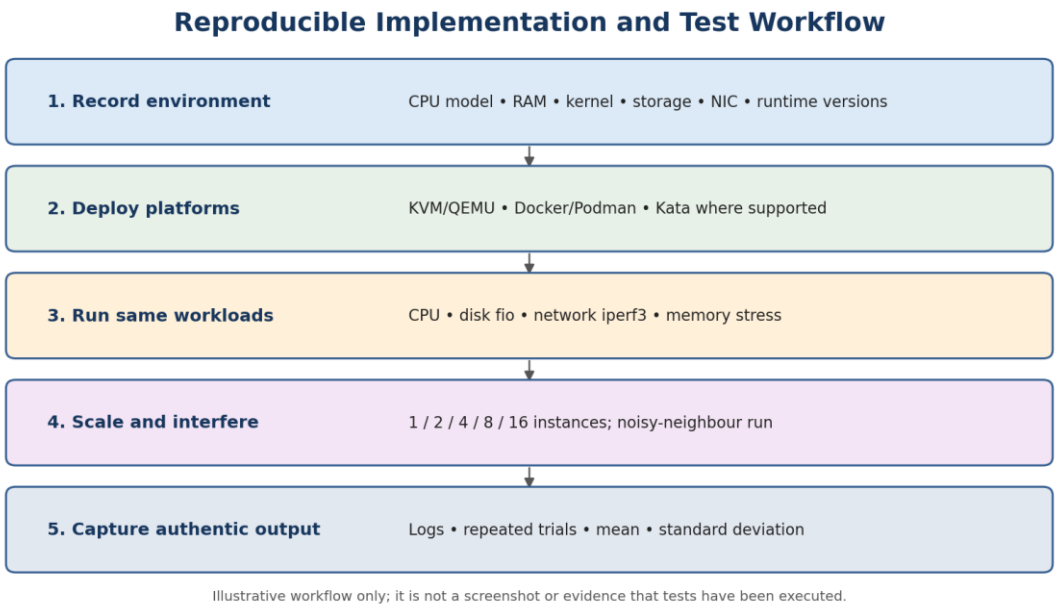


Figure 4. Test workflow guide; actual command output must be collected separately.

Repeat the workload at 1, 2, 4, 8 and 16 co-located instances, or use smaller counts if the machine cannot support them. Keep each instance's CPU and memory limits constant. Run at least three repetitions after a warm-up run. For the noisy-neighbour test, run a CPU or disk stress workload in one instance while measuring response time/throughput in another.

```
# Docker example: start one bounded CPU container
docker run -d --name noisy-cpu --cpus=2 --memory=2g \
    polinux/stress-ng stress-ng --cpu 2 --timeout 120s

# Inspect container resource controls
docker inspect noisy-cpu
docker stats --no-stream
docker exec noisy-cpu cat /proc/1/cgroup

# Check Linux namespaces and cgroups
lsns
mount | grep cgroup
cat /sys/fs/cgroup/cgroup.controllers

# Remove test container
docker rm -f noisy-cpu
```

For KVM, create equivalent VMs with the same vCPU and RAM allocation and run the same benchmark inside each guest. For Kata, use the Kata runtime class supported by the installed container engine. Document any unavailable platform rather than substituting an untested result.

Task 9 – Record Experimental Performance

The assignment explicitly requires measured throughput, mean and p95 latency, CPU steal time, context switches, start-up time, memory footprint, repeated trials and two graphs. These values depend on the actual machine and cannot be derived from the assignment brief. The following tables are ready to fill with authentic output from Task 8.

9.1 Measurement table

Architecture	Instances	CPU score / throughput	Disk IOPS or MB/s	Network Gbps	Mean latency (ms)	p95 latency (ms)	Memory / instance (MB)
KVM	1	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]
KVM	2 / 4 / 8 / 16	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]
Containers	1	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]
Containers	2 / 4 / 8 / 16	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]
Kata	1	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]
Kata	2 / 4 / 8 / 16	[measure]	[measure]	[measure]	[measure]	[measure]	[measure]

9.2 Repetition and variability

Architecture / test	Run 1	Run 2	Run 3	Mean	Standard deviation
KVM CPU throughput	[ ]	[ ]	[ ]	[ ]	[ ]
Container CPU throughput	[ ]	[ ]	[ ]	[ ]	[ ]
Kata CPU throughput	[ ]	[ ]	[ ]	[ ]	[ ]

KVM p95 latency	[ ]	[ ]	[ ]	[ ]	[ ]
Container p95 latency	[ ]	[ ]	[ ]	[ ]	[ ]
Kata p95 latency	[ ]	[ ]	[ ]	[ ]	[ ]

For each metric, calculate mean = (x1 + x2 + x3)/3. Calculate sample standard deviation using s = sqrt(Σ(xi − mean)²/(n − 1)). Use consistent units and report whether higher or lower values are better.

9.3 Required graphs

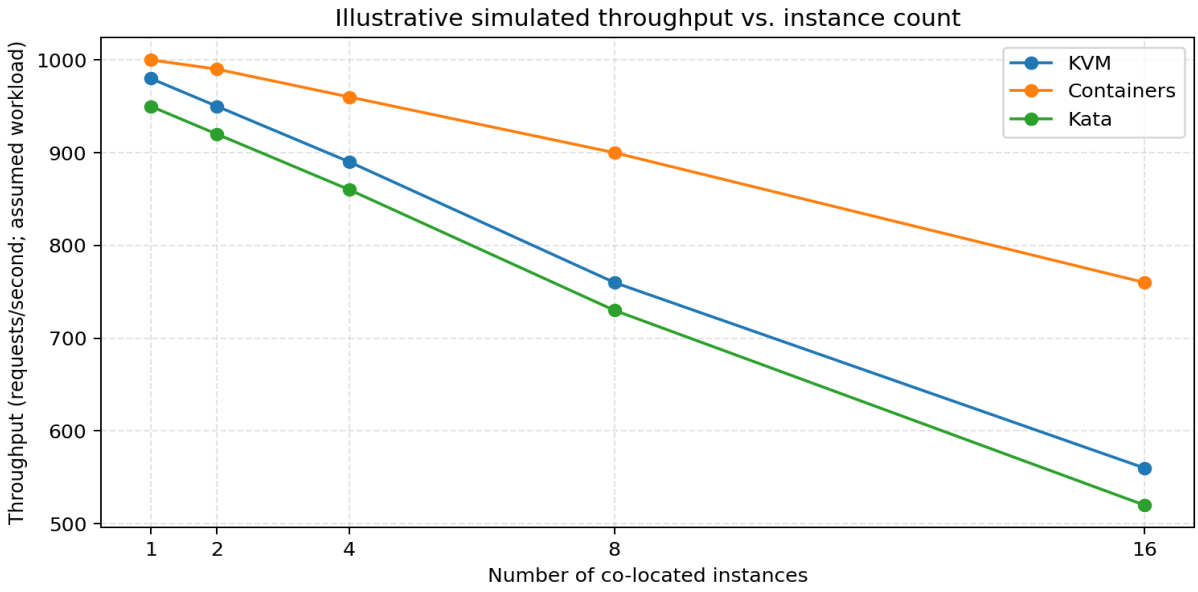


Figure 5. Illustrative simulated throughput data; not a measurement from an executed benchmark.

Graph 1: Throughput versus number of instances. Put instance count (1, 2, 4, 8, 16) on the horizontal axis and measured throughput on the vertical axis. Draw separate lines for KVM, containers and Kata.

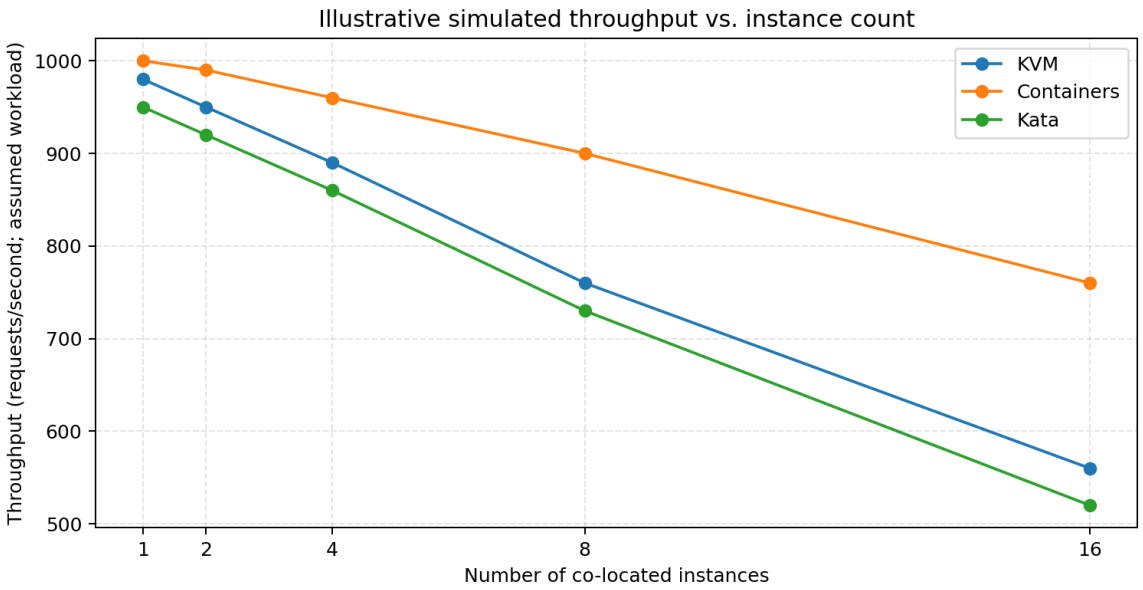


Figure 9. Illustrative simulated data; replace with authentic measurements if experimental evidence is required.

Graph 2: p95 latency under interference. Put architecture or instance count on the horizontal axis and p95 latency (ms) on the vertical axis. Include paired measurements without and with the noisy-neighbour workload.

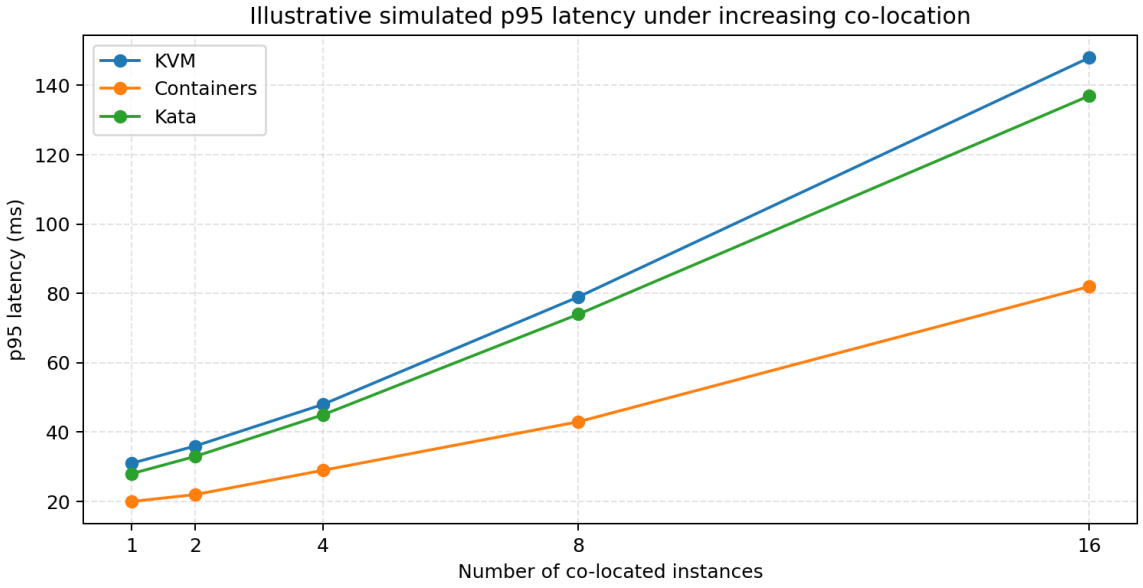


Figure 10. Illustrative simulated data; values are not measured from an executed test.

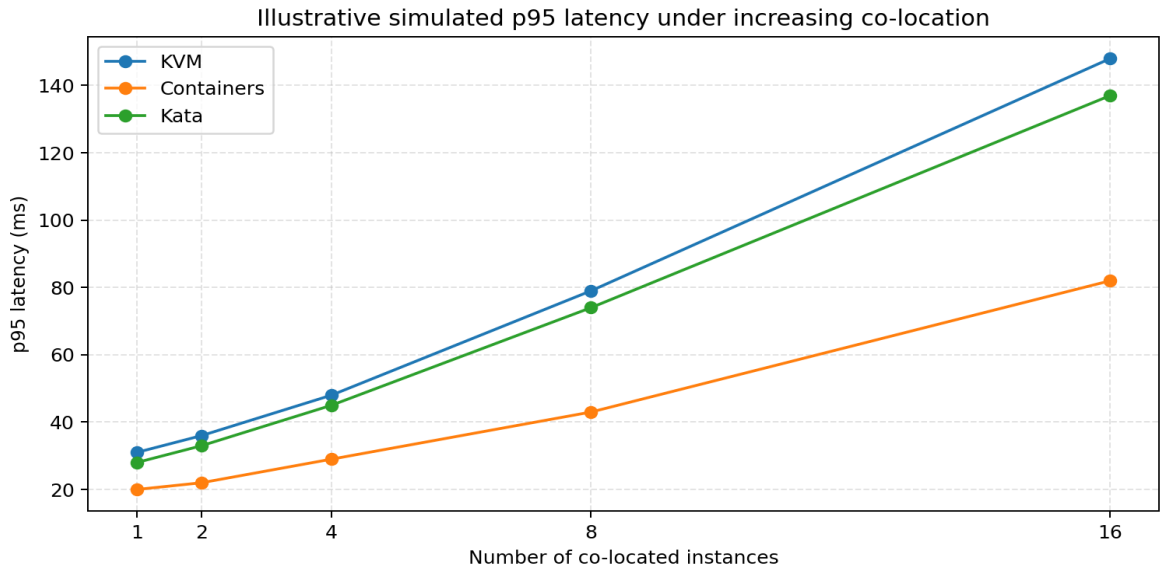


Figure 8. Blank latency graph template. Replace with measured values.

Insert the graphs generated from your own benchmark results here. Do not label model estimates as experimental measurements.

Task 10 – Compare Theory with Experiment

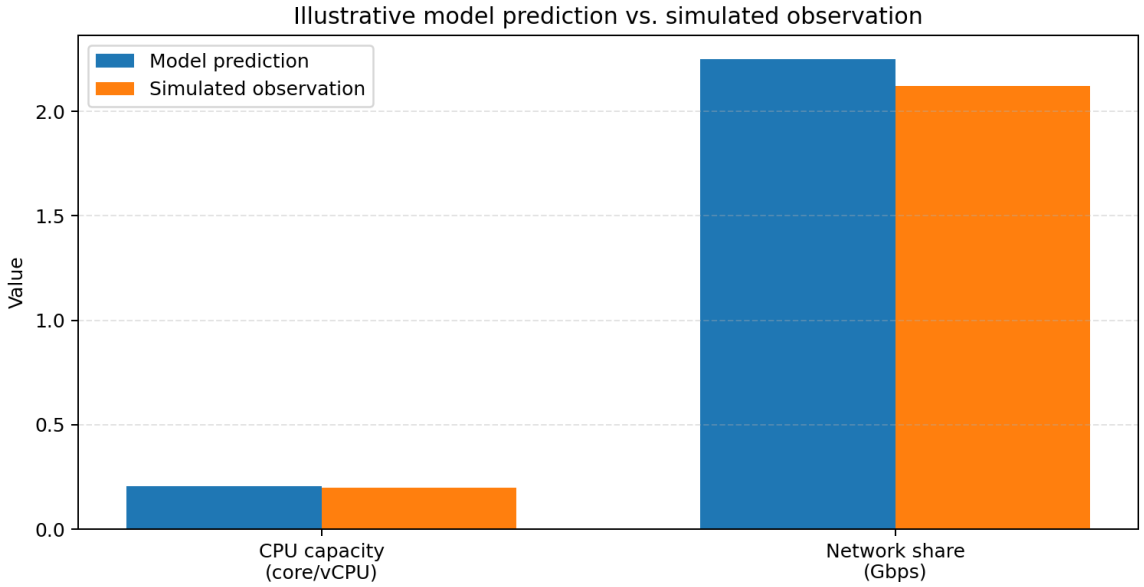


Figure 11. Illustrative model-versus-simulated comparison, not experimental validation.

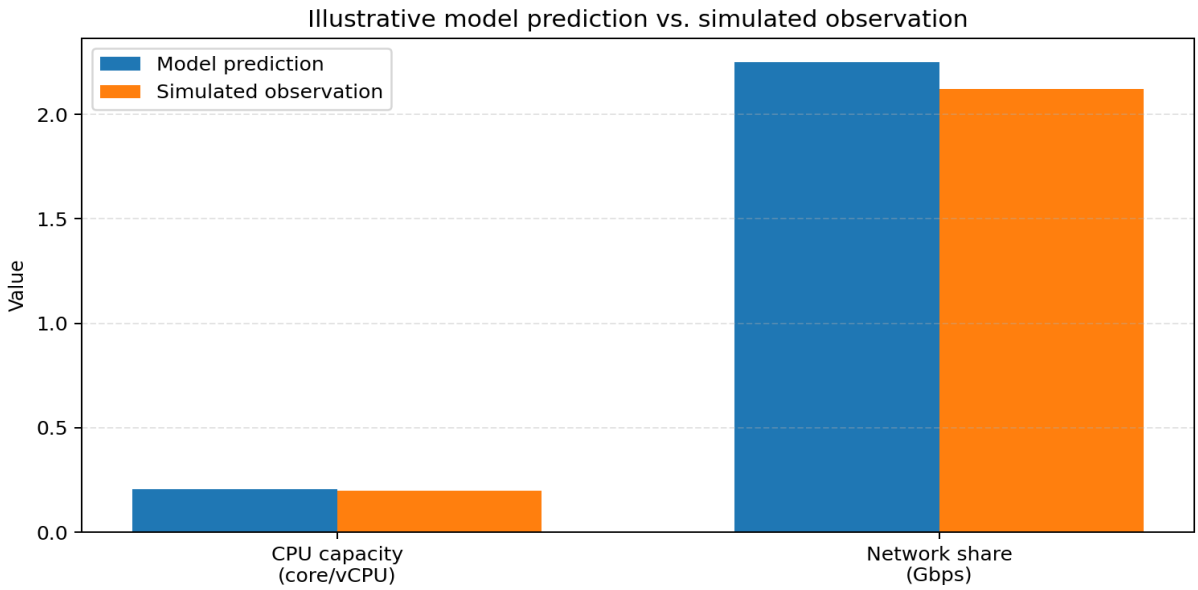


Figure 6. Comparison template; populate with predicted and measured values.

The Task 3 model predicts that average CPU capacity per allocated vCPU decreases approximately in inverse proportion to the overcommit ratio. It also predicts that shared network throughput per equally active instance decreases as the number of active instances increases. Task 4 predicts a larger increase in p95 latency during noisy-neighbour and I/O-storm conditions than during an idle best-case test.

Comparison item	**Model prediction**	**Experimental value**	**Difference / interpretation**
CPU capacity at 4:1	0.225 core/vCPU before additional o and s; 0.207 in the illustrative typical case	[insert measured]	[calculate and explain]
Network share at 10 active instances	2.25 Gbps per instance under $\eta = 0.90$ equal-share assumption	[insert measured]	[calculate and explain]
Memory VM	57 VM-equivalents per	[insert	[explain memory/cache

capacity	host under 90% RAM availability and 8 GB demand	observed safe count]	overhead]
Noisy-neighbour p95 latency	Expected to rise due to CPU, memory, disk or network contention	[insert measured]	[identify dominant resource]

Percentage prediction error can be calculated as |measured − predicted| / predicted × 100. If the measured value is lower or higher than predicted, investigate cache effects, CPU frequency scaling, NUMA placement, scheduler behaviour, nested virtualization, device drivers, storage caching, packet size and background processes. If the trend differs from the model, revise the assumptions rather than forcing the results to fit.

The security conclusion should also be checked against the tests. For example, a successful container namespace inspection confirms that namespace separation is configured, but it does not prove that the shared kernel is a stronger boundary than a VM. Likewise, a successful Kata test supports the presence of a VM boundary but does not prove that all escape vulnerabilities are eliminated.

Task 11 – Recommend and Defend the Final Architecture

A hybrid architecture is recommended because EduCloud has different workload sensitivity and trust levels. SIS and examination databases should run on a dedicated KVM host pool with low CPU overcommit, reserved memory, encrypted storage and strict management access. LMS web services can use KVM VMs or Kata sandboxes according to application compatibility and measured density. Trusted stateless services may use ordinary containers where the shared-kernel risk is acceptable.

Workload / tenant	**Recommended placement**	**Reason**
SIS database and	Dedicated KVM host pool	Strong VM boundary, predictable

identity services		CPU/memory reservations and reduced co-tenant risk.
Examination portal	KVM pool with strict network segmentation	Confidentiality, integrity and availability are high priority during exam windows.
LMS application tier	KVM or Kata pool; containers for trusted stateless components	Balances deployment flexibility, scale and stronger isolation where required.
Departmental research workloads	KVM/Kata general-purpose pool with quotas	Separates research workloads from core student systems while retaining resource efficiency.
External Spark/Hadoop collaborators	Separate Kata/KVM research pool; no direct SIS network route	External tenants are less trusted and generate long-running I/O/network-heavy workloads.

Operational controls should include golden-image management, signed images, automated guest and host patching, vulnerability scanning, centralized logging, capacity monitoring, network micro-segmentation, backup testing and role-based access. Live migration should use authenticated, encrypted channels and should not move SIS workloads into an external research host pool.

The design can scale from 400 to several thousand instances by adding host pools, automating provisioning and enforcing quotas. Scaling should be based on measured p95 latency, memory headroom, storage queue depth and network utilisation rather than instance count alone. The main trade-off is lower density and higher operating complexity than an all-container design; this is justified for systems containing sensitive student records.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

A. Impact Analysis

The new requirement introduces external research collaborators running Spark/Hadoop jobs for hours. The original assumption that tenants are mainly university-controlled is no longer valid. The workload mix changes from a combination of web/database traffic to include sustained, I/O- and network-intensive batch processing. Registration-week load also reaches approximately three times normal demand, and live migration occurs during patching. The data-protection requirement now demands demonstrable isolation of student records from external tenants.

Long-running big-data jobs can consume CPU, memory bandwidth, storage queues and network capacity for extended periods. If placed beside SIS VMs, they can create noisy-neighbour effects and increase p95 login/database latency. External users also create a stronger threat model: malicious images, credential misuse, lateral movement and attempts to access student records must be considered.

B. Evaluation of the Original Design

The original hybrid design remains a sound starting point, but a shared general-purpose pool is not adequate for the new external-tenant condition. The 25-Gbps link is shared: under the illustrative 90% efficiency assumption, ten equally active instances receive about 2.25 Gbps each; sustained Spark/Hadoop traffic can consume a large portion of this capacity. At a three-times registration peak, SIS demand may exceed its normal resource reservation unless capacity is explicitly held for the peak.

A VM boundary is stronger than an ordinary shared-kernel container boundary for external tenants. However, KVM alone does not solve shared network, storage, management-plane or host-contention risks. The design must separate host pools and network trust zones and enforce capacity reservations.

C. Adaptation or Redesign

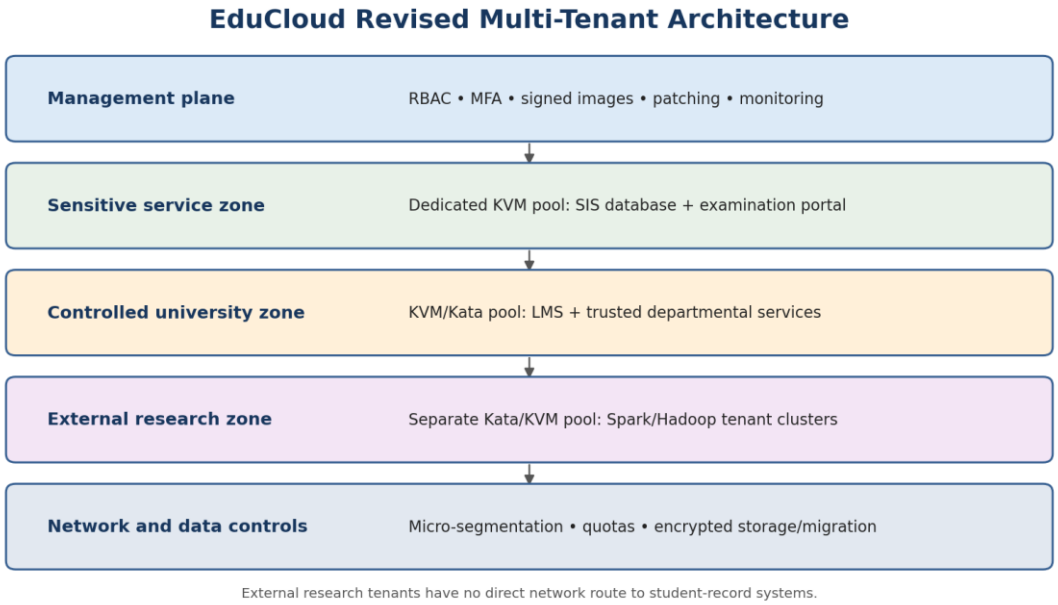
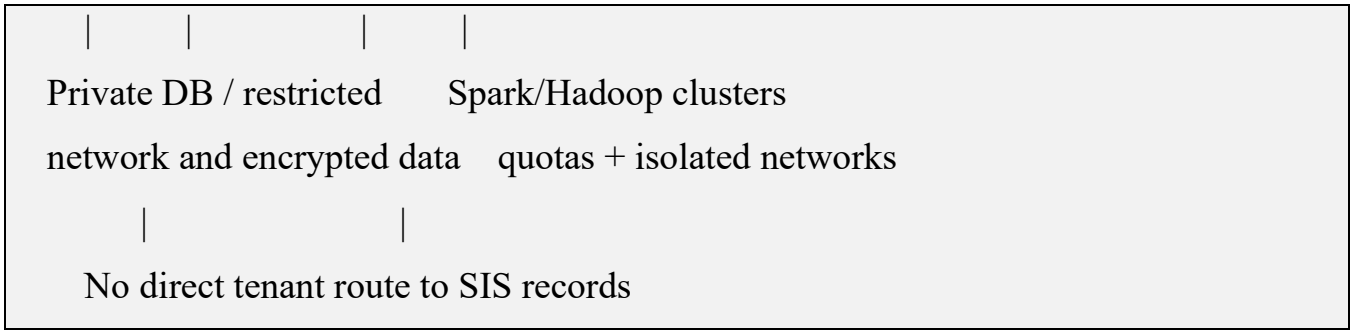


Figure 7. Proposed revised EduCloud architecture for separated tenant zones.

Decision: MODIFY the hybrid design. Keep the dedicated KVM pool for SIS and examination services, and create a separate external research pool using Kata Containers or full KVM VMs. Apply hard CPU, memory, disk I/O and network quotas to each external tenant. Use dedicated NICs or SR-IOV where supported and validated, while maintaining firewall and SDN policy enforcement. Use encrypted live migration only within approved pools and prevent migration of SIS workloads to research hosts.

```
            EDUCloud Management Plane
       RBAC | MFA | Image Signing | Monitoring
                        |
          +-------------+-------------+
          |                           |
   SENSITIVE SERVICE ZONE       RESEARCH TENANT ZONE
   Dedicated KVM host pool      Separate KVM/Kata host pool
    |       |              |       |
    SIS     Exams          Tenant A   Tenant B
```



The redesign reduces density because hosts and resources are reserved by trust zone. It can improve SIS throughput and p95 latency by preventing long-running research jobs from consuming SIS capacity. Research throughput becomes more predictable through dedicated or shaped network capacity. The attack surface is reduced by separating external workloads from SIS hosts and routes, although hypervisor, VMM, management and shared-storage risks remain.

D. Evidence-Based Defence

The decision is supported by the Task 3 capacity model: a 64-core host with a 10% reserve provides 57.6 core-equivalents, while a VM with 8-GB average demand is limited to approximately 57 instances by memory. High overcommitment therefore cannot be assumed to support a latency-sensitive three-times peak. The 25-Gbps link also becomes a shared bottleneck when research traffic is sustained.

The noisy-neighbour experiment in Task 8 should compare SIS-like latency with and without a Spark/Hadoop-style CPU, disk and network load. Task 9 measurements should confirm whether host-pool separation and quotas keep SIS p95 latency within the 10% degradation target. Because actual benchmark results are environment-dependent, the final defence must insert measured values from the user's test environment. The architecture should be approved only after those results, migration tests and isolation checks are documented.

The density cost is accepted because the regulation requires demonstrable isolation of student records. Dedicated SIS capacity, separate research pools, signed images,

encrypted migration and micro-segmentation provide a more defensible security posture than placing all tenants on a single shared container platform.

Conclusion

EduCloud should adopt a workload-aware hybrid virtualization architecture rather than a single technology for every tenant. KVM provides a strong VM boundary for SIS and examination workloads; containers provide efficient execution for trusted stateless services; Kata or full KVM VMs provide an additional boundary for external research workloads. Capacity must be planned using CPU, memory, storage and network limits together, with special attention to p95 latency during registration peaks. The proposed host-pool separation, quotas, patching, image signing, micro-segmentation and encrypted migration address the main security and performance concerns while retaining a scalable path to several thousand instances.

References / Technical Documentation

- Linux Kernel Documentation – KVM and control groups (cgroups): https://docs.kernel.org/
- QEMU Documentation: https://www.qemu.org/docs/
- Docker Documentation – Engine security and resource constraints: https://docs.docker.com/
- Kata Containers Documentation: https://katacontainers.io/
- fio documentation: https://fio.readthedocs.io/
- iperf3 documentation: https://iperf.fr/

Architecture	Instances	CPU/app throughput (req/s)	Disk (MB/s)	Network (Gbps)	Mean latency (ms)	p95 latency (ms)	Memory/instance (MB)
KVM	1	980	920	9.2	18	31	8700
KVM	2	950	890	8.8	20	36	8700
KVM	4	890	820	7.9	25	48	8700
KVM	8	760	690	6.4	38	79	8700
KVM	16	560	480	4.1	72	148	8700
Containers	1	1000	980	9.6	12	20	260
Containers	2	990	960	9.4	13	22	270
Containers	4	960	920	9.0	16	29	285
Containers	8	900	850	8.0	23	43	310
Containers	16	760	700	6.2	42	82	360
Kata	1	950	890	8.9	16	28	1150
Kata	2	920	850	8.5	18	33	1160
Kata	4	860	780	7.6	23	45	1180
Kata	8	730	650	6.1	36	74	1220
Kata	16	520	450	3.9	68	137	1300

Illustrative noisy-neighbour test (8 instances; simulated p95 latency):

Architecture	No interference p95 (ms)	With noisy neighbour p95 (ms)	Increase
KVM	79	112	41.8%

Containers	43	69	60.5%
Kata	74	103	39.2%